EduFuzzy: Intelligent Student Performance Risk Analyzer

PROJECT REPORT

Course Code and Course Name: ***Fundamentals of AI and ML, CSA2001***

Submitted By: ***Khushi Sehgal***

Roll Number: ***25BCE10421***

Date of Submission: ***30 March 2026***

Contents

1 Introduction **4**

2 Problem Statement **4**

3 Functional Requirements **4**

4 Non-functional Requirements **5**

5 System Architecture **5**

6 Design Diagrams **6**

6.1 Use Case Diagram 6

6.2 Workflow Diagram 6

6.3 Sequence Diagram 8

6.4 Class/Component Diagram 8

7 Design Decisions & Rationale **8**

8 Implementation Details **8**

9 Screenshots / Results **9**

9.1 Final Console Output 9

9.2 Risk Analysis Visualization 9

10 Testing Approach **9**

11 Challenges Faced 10

12 Learnings & Key Takeaways 10

13 Future Enhancements 11

14 References 11

1 Introduction

EduFuzzy is *an intelligent academic analytics system developed to move beyond traditional, rigid grading methods*. It leverages **Artificial Intelligence** techniques, specifically **Fuzzy Logic** and **Machine Learning** (Linear Regression and K-Means Clustering), to provide a holistic and proactive assessment of student performance risk. The system processes basic *academic metrics (Attendance and Internal Marks) to quantify risk, predict future performance, and segment students* into actionable groups, enabling timely intervention for those who are struggling.

2 Problem Statement

Traditional academic evaluation often relies on strict, discrete thresholds, failing to account for the **inherent uncertainty** and ambiguity in performance data (e.g., is 74% attendance 'Medium' or 'Low'? The line is often artificial). This rigidity leads to **delayed or inappropriate intervention** for at-risk students. The need is for a decision-support system that can:

1. Quantify subjective performance variables using soft computing (Fuzzy Logic).
2. Provide predictive insights (Linear Regression).
3. Automatically group students for targeted guidance (K-Means Clustering).

3. Functional Requirements

1. **Data Ingestion:** Must be able to load student data (Name, Attendance, Internal Marks) from a standard CSV file (`students.csv`).
2. **Fuzzy Risk Calculation:** Must use Attendance and Internal Marks as input to generate a quantified **Fuzzy Risk Score** (0-100) based on predefined rules.
3. **Grade Prediction:** Must predict a **Predicted Final Score** using Linear Regression based on Internal Marks.
4. **Student Clustering:** Must group students into distinct **Cluster Groups** (e.g., Safe, Monitor, Critical) based on their performance features using K-Means.
5. **Status Assignment:** Must assign a qualitative **Status** (Safe, Monitor, Critical) based on the calculated Fuzzy Risk Score.
6. **Reporting:** Must save the complete analysis (including all calculated scores and groups) to a new CSV file (`student_risk_report.csv`).
7. **Visualization:** Must display a bar chart visualizing the student names against their Fuzzy Risk Scores.

4. Non-functional Requirements

Table 1: Non-Functional Requirements Summary

Requirement	Description
Performance	The risk assessment and prediction must complete within a few seconds for a standard class size (e.g., up to 100 students).
Maintainability	The code must be modularized into distinct files (data_manager.py, fuzzy_variables.py, etc.) for easy updates.
Reliability	The Fuzzy Inference Engine must consistently apply the rules and produce reproducible risk scores for the same inputs.
Usability	The system must be executable via a single command (python main.py) and produce human-readable reports and charts.

5. System Architecture

The architecture now follows a Hybrid Inference Model. It comprises three main computational layers:

- **Fuzzy Inference Layer (Python)**: For quantitative risk scoring (0-100%).
- **Machine Learning Layer (Scikit-Learn)**: For predictive analytics and clustering.
- **Symbolic Logic Layer (Prolog):** A rule-based verification engine that uses Horn Clauses to cross-check critical risk cases against a strict knowledge base."

6. Design Diagrams

a. Use Case Diagram

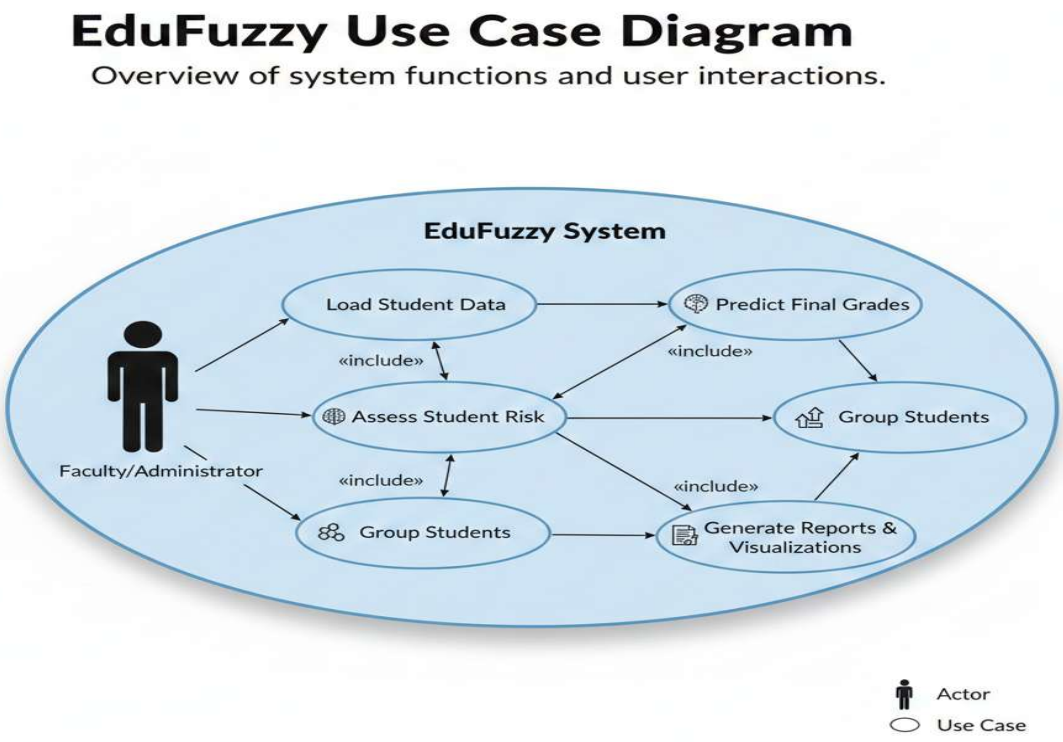


Figure 1: Use Case Diagram for the EduFuzzy System.

b. Workflow Diagram

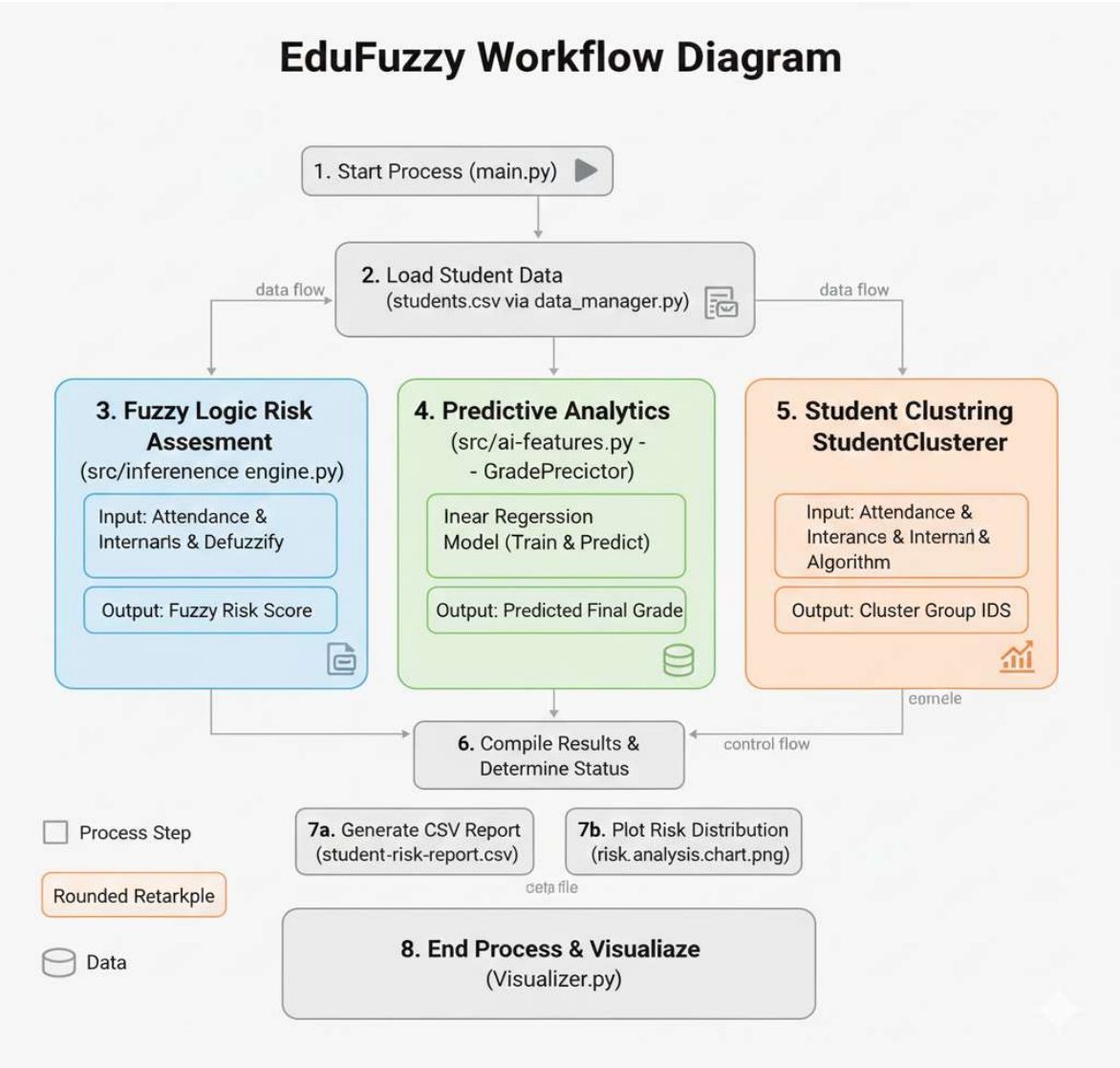


Figure 2: High-Level Workflow Diagram.

c. Sequence Diagram

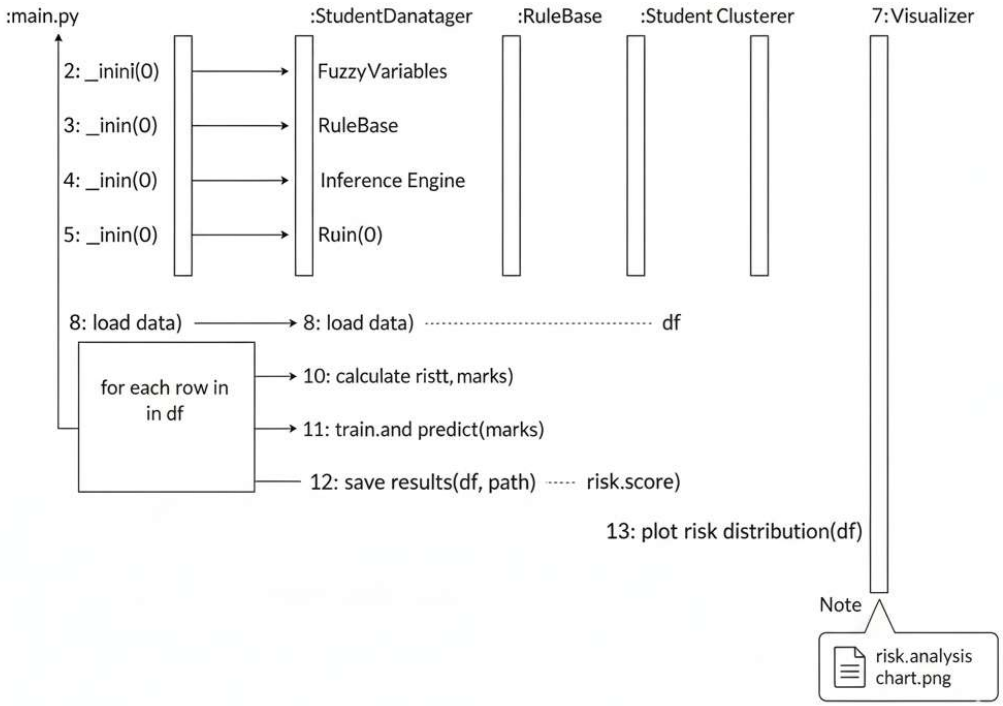


Figure 3: Sequence Diagram for Risk Calculation.

d. Class/Component Diagram

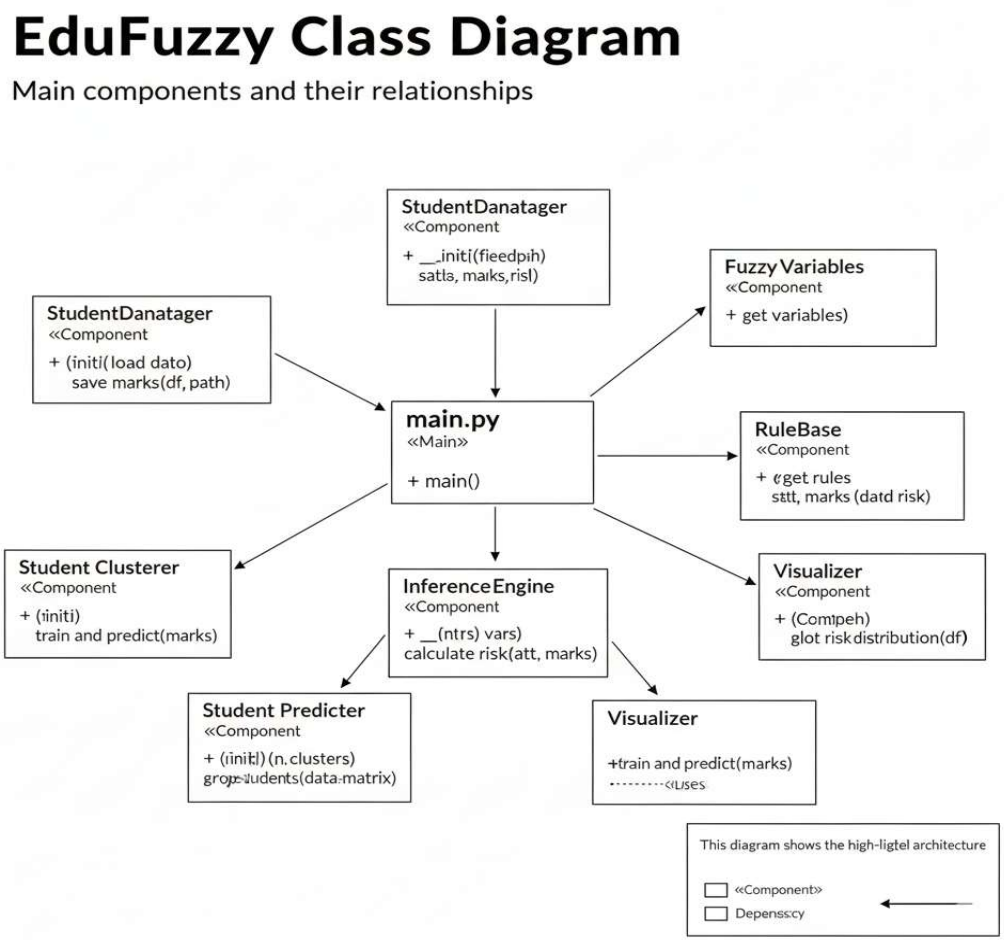


Figure 4: Class/Component Diagram.

7. Design Decisions & Rationale

- **Fuzzy Logic (skfuzzy):** Chosen to model the subjective, ambiguous nature of ”risk.” It handles partial membership (e.g., an attendance of 75% is not strictly ’medium’ or ’high’) better than rigid binary logic.
- **Linear Regression (sklearn):** Simple, interpretable model chosen for the predictive feature. It establishes a straightforward relationship between current marks and expected final score.
- **K-Means Clustering (sklearn):** An unsupervised learning technique used to automatically identify natural groupings of students based on their performance metrics, aiding in targeted intervention.
- **Modular Code Structure:** Separating functionality into files improves **maintainability**, **readability**, and allows for easy updates to specific components (e.g., changing the fuzzy rules without affecting the data loading).

8. Implementation Details

The project is implemented in **Python 3.x**. Key libraries used include Pandas, Scikit-Fuzzy, Scikit-Learn, and Matplotlib

Key Fuzzy Rules Implemented:

The Rule Base (src/rule_base.py) defines the core risk quantification logic:

- R_1: IF (Attendance is **low** OR Internal Marks is **poor**) THEN Risk is **critical**.
- R_2: IF (Attendance is **medium** AND Internal Marks is **average**) THEN Risk is **monitor**.
- R_3: IF (Attendance is **high** OR Internal Marks is **good**) THEN Risk is **safe**.

For Prolog (Propositional Logic):

- To demonstrate symbolic AI reasoning, a Prolog module (edufuzzy.pl) was implemented. It represents students as Facts and risk criteria as Rules."

  - **Code Snippet:**

```prolog
risk_status(A, M, 'Critical') :-
    attendance_low(A);
    marks_poor(M).
```

  - This module acts as a logic-based 'second opinion' to the fuzzy system.

9. Screenshots / Results

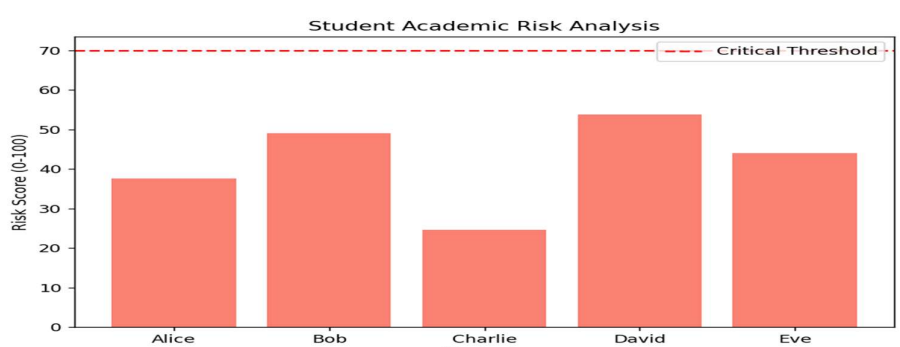
a. Final Console Output

The following is a representation of the final analysis data printed to the console:

```
Loading Data...
Data Loaded Successfully.
Calculating Fuzzy Risk Scores...
Running Linear Regression Prediction...
Running K-Means Clustering...

Final Analysis (Fuzzy + Regression + Clustering):
      Name  Attendance  Fuzzy_Risk_Score  Predicted_Final_Score  Cluster_Group   Status
0    Alice          85             37.56                   84.0              1  Monitor
1      Bob          60             49.05                   50.0              0  Monitor
2  Charlie          95             24.56                   96.0              1     Safe
3    David          40             53.77                   30.0              2  Monitor
4      Eve          75             44.05                   70.0              0  Monitor
Results saved to data/student_risk_report.csv
```

b. Risk Analysis Visualization



c. Prolog Logic Output

```
?- analyze_student(david)
|    .
Student: david (ID: 104)
  Attendance: 40%
  Marks: 15/50
  Risk Prediction: Critical

true
```

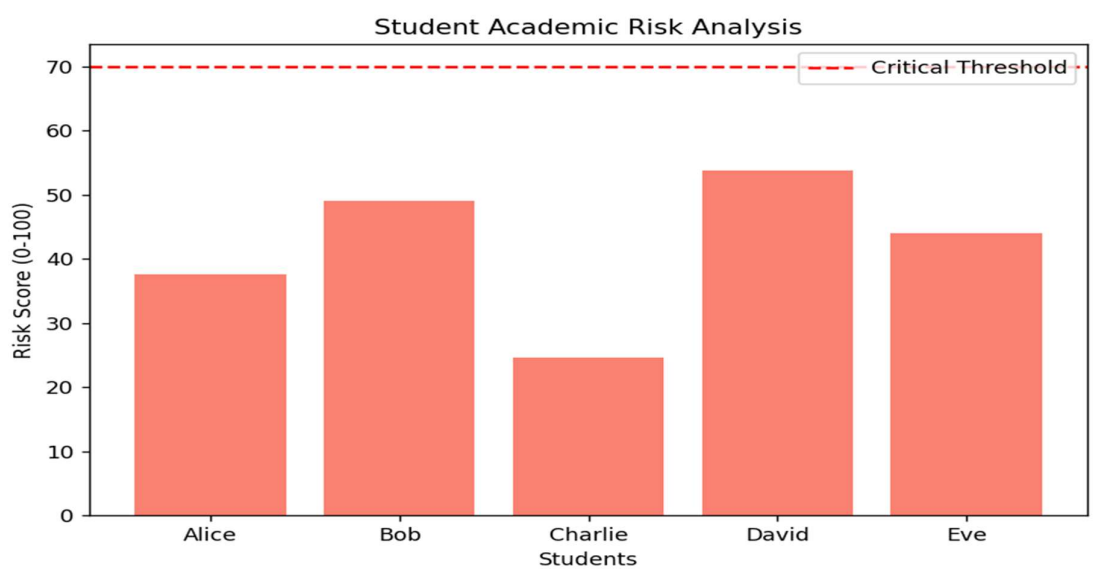
- Shows the output from the SWI-Prolog console, where the knowledge base is queried for a specific student's risk status.

10. Testing Approach

The system was tested using a **Unit Testing** approach, focusing on key functional components.

1. **Data Manager Test:** Verified that data_manager.py successfully loads students.csv and that the output report is saved correctly.
2. **Fuzzy Engine Test (Core Logic):** Manually tested edge cases (e.g., max risk: Att=20, Marks=10; min risk: Att=100, Marks=50) to ensure the calculate_risk function produced expected scores that aligned with the rules.

Figure 5: Generated Risk Analysis Visualization



3. **AI Feature Test:** Verified that the `GradePredictor` returns a predicted score and that the `StudentClusterer` assigns 3 distinct cluster labels.

11. Challenges Faced

- **Skfuzzy Integration:** Correctly defining the universe of discourse and the membership functions for the Antecedents (inputs) and Consequent (output) was initially challenging to ensure proper rule mapping.
- **Data Alignment:** Ensuring the Pandas DataFrames were correctly passed between modules and that column names were consistently used across `main.py` and `visualizer.py`.
- **AI Data Preparation:** Creating the dummy training data for the Linear Regression model required understanding how Scikit-Learn expects 2D array input for training and prediction (`.reshape(-1, 1)`).

12. Learnings & Key Takeaways

- **Fuzzy Logic:** Gained a practical understanding of how to implement soft computing to handle non-binary, ambiguous data, providing more nuanced insights than traditional logic.
- **Modular Programming:** Reinforced the importance of separating concerns in a project, making the code easier to debug and scale.

- **AI/ML Application:** Successfully integrated both supervised (Linear Regression) and unsupervised (K-Means) learning models into a cohesive, non-ML- specific application.

- **Symbolic vs. Numeric AI:** Learned the distinct advantages of both approaches—Fuzzy Logic is superior for nuance (scoring), while Prolog is superior for explainability and defining strict failure conditions.

- **Data quality and feature design strongly influence educational AI systems.**
  Building the pipeline from CSV input to risk scores, predictions, and clusters highlighted how crucial clean attendance/marks data and well-chosen features are for getting stable, interpretable results in educational analytics

- **Fuzzy logic makes grading fairer and more realistic.**
  While implementing EduFuzzy, it became clear that fuzzy sets and linguistic rules handle borderline and ambiguous student cases much better than rigid cut-offs, leading to more nuanced and fair academic risk evaluation.

13. Future Enhancements

1. **Adaptive Risk Thresholds:** Use a K-Means cluster analysis on the *Fuzzy Risk Scores* to dynamically define Critical, Monitor, and Safe bounds, instead of fixed thresholds.
2. **Expanded Input Variables:** Incorporate more factors like self-study hours, quiz scores, and course difficulty into the Fuzzy Inference Engine.
3. **Python-Prolog Bridge:** Implement 'pyswip' to allow the Python application to automatically send student data to the Prolog engine for real-time cross-verification.
4. **Web Interface:** Develop a simple front-end (using Flask or Streamlit) to allow faculty to upload data, view results, and visualize the membership functions interactively.

14. References

- Python Programming Language Documentation
- Scikit-Fuzzy Documentation https://scikit-fuzzy.readthedocs.io/
- Scikit-Learn Documentation https://scikit-learn.org/
- Github Repository: https://github.com/Sookchand/Fuzzy-Logic-Risk-Assesment
- Github Repository: https://github.com/HirudikaAnupama/Student-Score-Prediction-Linear-Regression
- Programming in Prolog: https://link.springer.com/book/10.1007/978-3-642-55481-0
- SWI-Prolog: https://www.swi-prolog.org/